

MEM++: NON-DESTRUCTIVE MEMORY FOR LONG-TERM ORGANIZATIONAL LLM AGENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Large Language Model (LLM) agents now take part in organizational work, where many authors record decisions across documents over months. Because a revised decision arrives as a new document rather than an edit, answering a question requires knowing which version held at a given time. However, most memory systems compress the record at write time. By distilling each document into facts, notes or graph edges, these methods fix what can be answered before any question is asked. To address this, we propose Mem++, a non-destructive memory framework shifting from write-time distillation to read-time selection. Mem++ stores every document whole with its date and author, and it calls no generative model at write time. At read time, it retrieves only documents dated up to the time a question asks about and fuses lexical and semantic rankings. Unlike systems that overwrite older versions, Mem++ keeps them and leaves the choice to the answering model. Evaluations on the organizational benchmark OrgMemBench demonstrate that Mem++ surpasses the strongest memory system baseline by 8.0 to 13.1 points across two answering models. With gpt-4.1-mini, it also achieves the best overall score, 2.6 points above RAG. In addition, Mem++ achieves the best average LLM-judge score on LoCoMo and ranks second on LongMemEvals, behind only its entity-graph variant. Code will be released upon publication.

1 INTRODUCTION

Large Language Model (LLM) agents now take part in organizational work, from daily tasks to decisions that span many teams over months (Kirchdorfer et al., 2026). Such critical work is only possible if the agent can correctly recall what was already decided (Zeng et al., 2025). In an organization, the record of past decisions is shared; organizational memory keeps the history that informs present decisions (Walsh & Ungson, 1991), and it is written by humans and agents alike. To have a deeper understanding of the context and intent of the organization, a shared memory that mixes agents, humans, and compound their knowledge becomes a necessity.

Organizational memory differs from the conversational, single-agent memory that existing systems were built for (Figure 1). Conversational memory has a single dialogue whose statements accumulate rather than compete, and a later statement is therefore read as an update of an earlier one. Organizational memory, in contrast, is written by many people independently of one another across email, tickets, meeting notes and shared documents (Gardner, 2026). Moreover, a decision that changes an earlier one is written as a new document rather than as an edit to the old one, and the earlier version stays in the record with its own date and its own author. For example, a policy set in March, revised in June and reversed in September exists as three documents, and all three remain because nobody goes back to remove them. Consequently, answering a question against this record takes more than finding the most recent version. This requires knowing when each version held, who wrote it. Conversational memory struggles here, because they treat every statement as coming from the same authority. For example, Mem0(Chhikara et al., 2025) resolves the conflict by overwriting, so the approved decision is replaced by the proposal and can no longer be retrieved afterwards. In addition, Zep (Rasmussen et al., 2025) keeps both statements, because its Graphiti engine stamps every edge with the valid period, but it stores extracted facts rather than the documents.

Existing memory systems resolve one trade-off at ingest, between keeping the record and structuring it. Traditional retrieval-based systems keep the raw text and search it by similarity (Packer

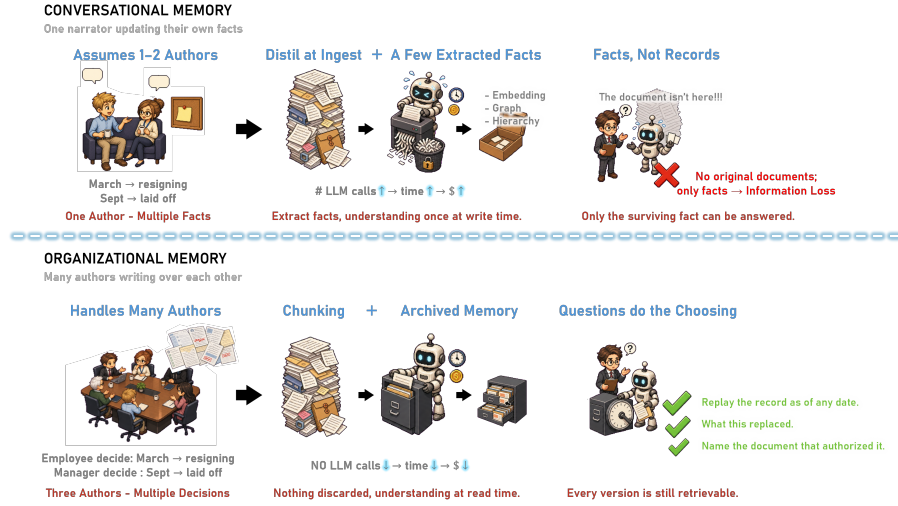


Figure 1: **Conversational vs. organizational memory.** **Top:** a single narrator restates their own facts, and LLM extraction at ingest keeps only the surviving fact. **Bottom:** different authors write the same fact, and an earlier decision may still hold. Mem++ stores each original document with no generative model call at write, and every version stays retrievable.

et al., 2023), and nothing is discarded, but nothing is organized either. A query, therefore, cannot distinguish a superseded value from one still in force. This is how the memory stream of generative agents works, appending each observation in its original form and retrieving it later by relevance and recency (Park et al., 2023). Structured systems make the opposite choice. They record how statements relate to one another at ingest, and the raw text does not survive that step. For instance, Mem0 stores a set of facts (Chhikara et al., 2025), Zep a graph of dated edges (Rasmussen et al., 2025), and HippoRAG and GraphRAG an entity graph extracted from the corpus (Gutiérrez et al., 2024; Edge et al., 2024), and in none of them does a later query reach the document itself. A memo that changes a decision is reduced to a relation between an entity and a value. Its reason, condition and approval are not entity-shaped, and they never enter the store. What will be answerable is therefore fixed before any question has been asked. As we show in Section 4, this explains the sharp drop of conversational memory systems on OrgMemBench, a benchmark of multi-author temporal, contradiction and justification-chain questions that define organizational memory.

To address these limitations we introduce **Mem++**, an organizational memory architecture that moves the decision about what matters from write time to read time. (i) Every artifact is stored whole and remains the unit of retrieval. (ii) Structure can be added as an optional layer over the retained documents rather than as a replacement for them. A Mem++ pass groups near-duplicates and marks older conflicting versions as superseded without deleting them, and we evaluate it as an ablation. (iii) Selection is deferred until a question arrives, and it therefore ranges over the full record rather than a pruned set of facts. Each retrieval leg is bounded by date, and a question about a past state is answered only from documents written up to that time. What distinguishes Mem++ is therefore not that the record is kept, but that each document stays whole with its date and the choice between versions is made at read time. Our contributions are as follows:

- **Mem++, a non-destructive organizational memory.** We introduce a memory that stores each document whole with its date and author, calls no generative model at write time, and defers the choice between versions to read time.
- **Evaluation.** We evaluate Mem++ against publicly available memory systems on the recently released organizational benchmark OrgMemBench (Gardner, 2026) and on the conversational benchmarks LoCoMo and LongMemEval_s, where it outperforms every memory baseline on OrgMemBench and remains competitive on the conversational benchmarks.
- **Ablations.** We measure the contribution of each retrieval leg, toggle consolidation and extracted-fact cards, and compare with the entity graph of Mem^{g++}.

2 RELATED WORK

2.1 CONVERSATIONAL MEMORY ARCHITECTURES FOR LLM AGENTS

Agent memories give an LLM persistent state beyond its context window, differing in how they carry information across sessions. MemGPT (Packer et al., 2023) and the generative agents memory (Park et al., 2023) hold the record in full and leave every judgement to retrieval. Details survives, but nothing marks how one statement stands to another, so a query cannot separate a replaced value from one still activated. Retrieval-augmented generation (RAG) grounds answers in an external corpus by chunking documents, embedding them, and returning the nearest chunks at query time (Lewis et al., 2020; Karpukhin et al., 2020). However, the words survive while the document does not, since chunking separates a passage from the rest of its text and from any later version of it. Most recent approaches instead compress at ingest, gaining structure at the cost of the record. Mem0 distills each document into facts and overwrites them on conflict (Chhikara et al., 2025). Similarly, A-Mem, Nemori, LightMem and LiCoMemory rewrite interactions into episodes through clustering, summarization or note-taking (Xu et al., 2026; Nan et al., 2025; Fang et al., 2026; Huang et al., 2026). In addition, G-Memory organizes these episodes hierarchically (Zhang et al., 2026), PREMem reasons as it writes (Kim et al., 2025), and MemOS exposes memory as a managed layer (Li et al., 2025). Each gains a compact store but loses the source and the links between its statements, and what can be answered is fixed at ingest. In contrast, Mem++ stores every document whole at write time and marks superseded statements with a date instead of deleting them. This approach keeps every version available and leaves the choice between them to read time, and conflicting statements from different authors therefore remain answerable.

2.2 LONG-TERM TEMPORAL MEMORY SYSTEMS

Temporal questions ask what held at a given time or what a newer statement replaced, and conversational benchmarks such as LongMemEval and LoCoMo test them through knowledge-update and temporal-reasoning categories (Wu et al., 2024; Maharana et al., 2024). Existing systems handle such changes in two ways. The first discards or alters the older value. For instance, Mem0 overwrites an older fact once a newer one arrives (Chhikara et al., 2025), A-Mem revises related notes in place when a new one is added (Xu et al., 2026), and MemoryBank lets older memories fade along a forgetting curve (Zhong et al., 2024). The second keeps the older value with a date. Zep and LongMemEval attach time intervals or timestamps to stored facts so that queries can be bounded by date (Rasmussen et al., 2025; Wu et al., 2024). In conversational memory, both mechanisms treat a change as one narrator updating their own facts, and the newest value is therefore assumed to be correct while the older one is removed or kept only as history. In an organization, however, changes arrive as new documents from different authors across different channels, and questions ask about past versions as often as about the current state (Gardner, 2026). Mem++ addresses this by keeping every version as a stored document, marking the replaced one with the date it stopped holding. Then, each retrieval is bounded by the date in the question, which lets one store answer what holds now, what held before, and what changed. In other terms, it reconstructs the life journey of a fact and how it changed. This is useful for organization to track pivot points in decisions making, allowing the agents and humans to revisit these later.

3 METHOD

Organizations keep records to preserve their decisions over time (Walsh & Ungson, 1991). Unlike a conversation, this record is a stream of dated documents from many authors. A useful memory must therefore tell what the current value is, what the value was at a past date, and what a change replaced and who made it. To this end, Mem++ stores each new document whole, never rewrites or deletes one to resolve a conflict, and leaves the choice between versions to the answering model. Figure 2 shows its three stages, namely a write path that stores documents, an optional consolidation pass that links their versions (Section A), and a read path that answers a question against the store. Algorithm 1 in Section A gives the pseudocode.

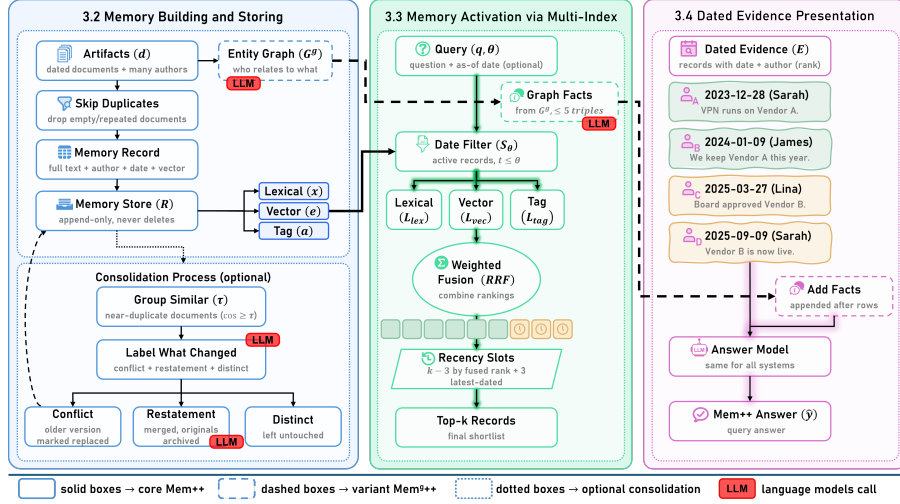


Figure 2: Overview of Mem++ framework. (3.2) Each document is stored whole with its author, date and sentence embedding, and it is indexed three ways. (3.3) Records valid at the as-of date θ are retrieved from the three indexes and fused with weighted RRF, and three of the k matches for the latest-dated records. (3.4) The top- k records are passed with their dates and authors to a fixed answering model. Mem^g++ graph triples carry no date.

3.1 ARCHITECTURE

Formally, Mem++ consists of one store and two operators,

$$\text{Mem++} = \langle \mathcal{R}, \mathcal{C}, \Phi \rangle, \quad (1)$$

where $\mathcal{R}$ is the memory store filled by the write path, $\mathcal{C}$ is an optional consolidation operator that relates versions, and Φ is the read operator that retrieves evidence for a question. Each stored document becomes one record, or row,

$$r = \langle x, a, t, e, \sigma \rangle, \quad (2)$$

where x is its full text, a is its author, t is its event date, e is an embedding of x , and σ is its state, which records whether the row is active and which later row, if any, superseded it. Only σ may change after a row is written, and no row is ever deleted. Hence, the store $\mathcal{R} = \{r_1, \dots, r_N\}$ only grows, and every earlier version remains in the store with its original text. The consolidation operator $\mathcal{C}$ groups near-duplicate rows and marks older conflicting versions as superseded in σ . It is described in Section A, disabled in all headline runs and evaluated in the ablations.

3.2 MEMORY BUILDING AND STORING

Unlike approaches that segment the stream into windows or rewrite it into extracted units (Chhikara et al., 2025), the write path stores each document once and as a whole, so no segment boundary splits a document and no model guesses at write time what a later question may need.

Whole-Document Ingestion. For each document, the write path creates one row r of the form in Eq. equation 2, with embedding $e = h(x)$ from a sentence encoder $h(\cdot)$ (Table 6), and sets σ to mark the row as current and active. The text is indexed both in a full-text index for lexical search and through e for vector search, and no generative model is called on this path. Documents longer than the encoder limit are truncated only in e , while x and the lexical index keep the full text.

Provenance and Time Anchoring. The author is recorded as a tag wherever the source names one, and this tag feeds the tag index. On OrgMemBench the benchmark loader passes no author field, so this tag stays empty for every system, although names appear inside the text. The event date is stored in t whenever the source provides one, as OrgMemBench documents do (Gardner, 2026). On the conversational benchmarks, each turn is stored with its session date in t , and the date and speaker also lead its text.

Append-Only Incremental Updates. New documents are added without touching the rows already stored. The write path skips a document whose text is empty or exactly repeats an active row in the same author scope, so re-ingesting the same stream adds no duplicate current rows. Adding a document thus costs one embedding, and no older row is reprocessed.

3.3 RETRIEVAL VIA MULTI-INDEX FUSION

Given a question q and an optional as-of date θ , the read path selects the k rows that the answering model will see:

$$E = \Phi(q, \theta \mid \mathcal{R}) = \text{TopK}(\text{RRF}(L_{\text{lex}}^{\theta}, L_{\text{tag}}^{\theta}, L_{\text{vec}}^{\theta})), \quad (3)$$

where L_{ℓ}^{θ} is the ranked list that index $\ell \in \{\text{lex}, \text{tag}, \text{vec}\}$ returns for q within the temporal scope defined below, and TopK keeps k rows as described below. Because questions range from exact identifiers to loose paraphrases, Mem++ queries three indexes over one shared scope and fuses their rankings without a language model call.

Temporal Scoping. All three indexes draw from the same base set of rows,

$$\mathcal{S}_{\theta} = \{r \in \mathcal{R} \mid \text{active}(\sigma_r), t_r \leq \theta\}, \quad (4)$$

where the time condition is dropped when the question carries no as-of date. Because this condition is applied before any index ranks a row, all k slots go to eligible rows, whereas filtering after ranking could leave fewer than k . Rows without an event date fail the time condition, so an as-of question can only reach dated rows.

Lexical, Tag, and Semantic Pathways. Within $\mathcal{S}_{\theta}$, each index returns its own ranked list of at most C candidates. The lexical index matches the content words of q against the full-text, and ranks matches by cover density, recovering exact identifiers and an embedding may blur. The tag index matches q , against author tags; since the reported configurations carry only author labels, it rarely fires and is evaluated as an ablation. The semantic index ranks rows by the cosine similarity between $h(q)$ and e , recovering paraphrases that share no words with the question.

Weighted Rank Fusion with Recency Reserve. The three lists are merged with weighted reciprocal rank fusion (Cormack et al., 2009), which scores a row by

$$\text{RRF}(r) = \sum_{\ell \in \{\text{lex}, \text{tag}, \text{vec}\}} \frac{w_{\ell}}{\gamma + \text{rank}_{\ell}(r)}, \quad (5)$$

where w_{ℓ} is the weight of index ℓ , $\text{rank}_{\ell}(r) \in \{0, 1, \dots\}$ is the position of r in L_{ℓ}^{θ} , γ is a smoothing constant, and a row absent from a list receives nothing from it. Because fusion uses ranks rather than raw scores, the three pathways need no score normalization. Ties are broken by ingest recency. Of the k returned rows, $k - 3$ follow this order, and the other three go to the remaining matched rows with the latest event date, falling back to ingest time where no event date exists. If fewer than three matched rows remain, the reserve is filled with the most recently ingested unmatched rows. Read operator Φ does not read the supersession links in σ , so a superseded row competes like any other active row.

Graph Retrieval (Mem^{g++}). At ingest, Mem^{g++} extracts entity-relation triples from each document with a language model, and these triples form the graph. At query time, one model call extracts the entities named in, and the graph is searched from those entities to collect related triples. These graph results are appended after the fused rows, at most five per question, so Mem^{g++} sees more context than the systems it is compared against. Triples carry no date and the as-of condition is not applied to them, so on a dated question the graph can surface relations from artifacts later than θ .

Graph Retrieval (Mem^{g++}). Mem^{g++} adds to Mem++ the entity-relation graph of Mem0^g (Chhikara et al., 2025). At write time, two gpt-4o-mini calls extract entity-relation triples from each document, and these triples form the graph $\mathcal{K}$. At query time, one model call extracts the entities V_q named in q , and $\mathcal{K}$ is searched from those entities to collect related triples. These triples

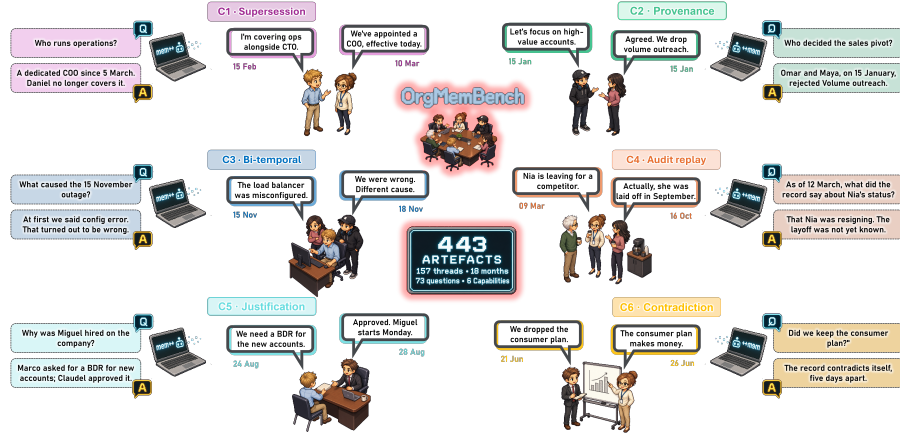


Figure 3: The six capabilities of OrgMemBench (Gardner, 2026), with questions and answers examples. The tier we evaluated holds 443 artifacts across 157 threads for 18 months, with 73 questions.

are appended after the fused rows, at most five per question. This is why on the conversational benchmarks, Mem^{g++} sees more context than the systems it is compared against. The triples carry no date and the as-of condition is not applied to them, so on a dated question the graph can surface relations drawn from documents later than θ .

3.4 EVIDENCE PRESENTATION

Each row is rendered in rank order as its original text preceded by its event date and, where the source records one, an author tag, so that the model can compare versions of a fact by when they were written. On OrgMemBench the header is built from t and that tag, although no artifact on the reported tier names an author, so every row’s header carries the same placeholder author alongside the date. On the conversational suites the date and speaker already lead the stored text, so each row is shown as stored, preceded only by its rank number. This design keeps Mem++ a memory rather than a reasoner, so it adds no inference step of its own at answer time.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Datasets. We evaluate Mem++ on three benchmarks, one organizational and two conversational, which together cover the settings it is Mem++ designed for. **OrgMemBench** (Gardner, 2026) is our primary benchmark, a synthetic bi-temporal corpus of one fictional organization that we use unmodified. We run its medium tier, which pairs 443 dated artifacts, organized into 157 threads over 18 months where later decisions revise earlier ones, with 73 questions over six capabilities that range from supersession to contradiction (Figure 3). The question count is small, yet each answer is graded against a rubric rather than marked right or wrong, giving 247 graded facets and a score in $[0, 1]$ that reflects partial coverage of the expected evidence. **LoCoMo** (Maharana et al., 2024) provides 10 multi-session dialogues of about 24K tokens each and 1,540 questions across four reasoning categories, and it is the most widely used for memories. **LongMemEvals** (Wu et al., 2024) keeps a similar question format while spreading each of its 500 questions over roughly 48 sessions and 494 turns, which tests if a memory still holds when the history grows.

Baselines. We benchmark against seven methods that range from no external memory to richly structured ones, namely **Full Context** (entire history in the prompt), **RAG-4096** (Lewis et al., 2020, 4096-token chunks for dense retrieval), **Mem0** (Chhikara et al., 2025, fact extraction with update and delete rules), **Zep** (Rasmussen et al., 2025, temporal knowledge graph with dated edges), **A-Mem** (Xu et al., 2026, structured notes with evolving links), **LangMem** (LangChain AI, 2025, automatic memory extraction across sessions), and **gbrain** (Gardner, 2026, entity graph over organizational

Table 1: Performance on OrgMemBench (Gardner, 2026) categorized by question type. Overall is weighted by the number of questions in each category and reported with its standard deviation across runs. Bold indicates the best performance. Underline indicates the second best performance.

	Method	Supersession	Decision Provenance	Bi-temporal	Audit Replay	Justification Chain	Contradiction	Overall
gpt-4.1-mini	Full Context	0.0	27.5	0.0	33.0	22.7	8.3	17.8±0.1
	RAG	66.1	79.7	66.7	40.9	19.6	75.4	55.0±0.3
	Zep	31.4	42.5	23.0	76.8	20.7	43.3	41.0±0.2
	Mem0	40.8	55.0	50.0	25.1	14.4	<u>52.8</u>	36.9±0.0
	A-Mem	58.3	63.3	43.6	36.5	20.7	43.5	44.5±0.3
	gbrain	54.8	60.7	30.3	48.5	22.7	22.9	43.2±0.1
	Mem++	67.7	<u>65.9</u>	50.0	83.0	<u>21.5</u>	47.2	57.6 ±0.1
	Mem ^g ++	<u>67.2</u>	63.9	<u>50.6</u>	<u>79.7</u>	19.8	44.4	<u>55.9</u> ±0.1
	Full Context	2.5	21.1	0.0	35.7	17.0	13.9	16.8±0.1
	RAG	<u>56.7</u>	47.8	30.2	55.1	<u>10.4</u>	67.2	43.4±0.2
gpt-4o-mini	Zep	27.5	38.3	34.9	74.9	7.4	29.6	36.2±0.3
	Mem0	32.3	47.2	<u>33.3</u>	32.4	11.8	62.8	33.8±0.1
	A-Mem	42.8	60.0	28.6	21.3	4.4	41.7	32.6±0.3
	gbrain	47.8	46.1	25.9	46.6	7.7	21.9	34.7±0.2
	Mem++	57.0	<u>49.2</u>	<u>33.3</u>	70.7	8.9	39.8	<u>44.2</u> ±0.1
	Mem ^g ++	<u>56.7</u>	48.6	32.1	<u>71.9</u>	8.8	37.7	44.4 ±0.1

documents). Full Context, RAG-4096, Mem0 and Zep serve as general-purpose baselines and run on all three benchmarks, whereas A-Mem and LangMem, which designed for conversational memory, run only on LoCoMo and LongMemEvals, and gbrain, which was released with OrgMemBench, runs only there.

Evaluation Metrics. On all three datasets, our primary evaluation metric is the **LLM-judge score** (abbreviated as **LLM** for simplicity), using gpt-4o-mini as the judges. On LoCoMo and LongMemEvals, we additionally report **F1** and **BLEU-1**. For OrgMemBench, the judges grade against the benchmark’s rubric. For LongMemEvals, prompts are adapted to its task-specific question-answering format, following Zep (Rasmussen et al., 2025). These metrics are accuracy metrics of different standard, and are scaled to the 0–100 range, with higher values indicating better performance and 100 denoting a perfect score.

Implementation Details. To ensure fair comparison, Mem0 and Zep retrieve memory contexts, which are then fed to gpt-4o-mini, gpt-4.1-mini for answer generation. All other methods, including Mem++, employ the same models for answer generation, while Mem^g++ uses gpt-4o-mini as its internal graph backbone. All retrieval-based baselines employ the powerful text-embedding-3-small model, whereas Mem++ uses the lightweight all-MiniLM-L6-v2 model. The lexical index matches query terms disjunctively by default and conjunctively on OrgMemBench. Key hyperparameters are the fusion constant γ and the pathway weights w_ℓ , while the consolidation threshold τ applies only when consolidation is enabled. For retrieval count, we fix $k = 50$ in main experiments, while k varies from 10 to 100 in the ablation study, as shown in the Appendix.

4.2 MAIN RESULTS

Tables 1, 2 and 3 report the results on OrgMemBench, LoCoMo and LongMemEvals. Regarding these tables, we highlight the following observations.

Strong Performance on Organizational Memory. Mem++ achieves the best overall score on OrgMemBench with gpt-4.1-mini, reaching 57.6 and surpassing the strongest baseline, RAG, by 2.6 points. With gpt-4o-mini, Mem^g++ and Mem++ rank first and second with 44.4 and 44.2, ahead of RAG at 43.4. The memory systems fall further behind, and Mem++ exceeds the best of them by 13.1 points over A-Mem with gpt-4.1-mini and by 8.0 points over Zep with gpt-4o-mini. Full Context performs worst, scoring below 18 overall and near zero on Supersession and Bi-temporal questions with both answerers. This suggests that placing the whole record in context does not by itself reveal which version holds.

Supersession and Audit Replay. Mem++ is the best method on Supersession with both answerers, scoring 67.7 and 57.0. It also leads Audit Replay with gpt-4.1-mini at 83.0, which is 6.2 points above Zep, while Mem^g++ leads it with gpt-4o-mini at 71.9. We attribute the gap over Mem0 and

Table 2: Performance on LoCoMo (Maharana et al., 2024) categorized by type. Bold, underline indicates best and second best performance. The baseline performance comes from Nan et al. (2025).

Method	Temporal Reasoning			Open Domain			Multi-Hop			Single-Hop			Average			
	↑LLM	↑F1	↑BLEU	↑LLM	↑F1	↑BLEU	↑LLM	↑F1	↑BLEU	↑LLM	↑F1	↑BLEU	↑LLM	↑F1	↑BLEU	
gpt-4.1-mini	Full Context	74.2	47.5	40.0	56.6	28.4	22.2	77.2	44.2	33.7	86.9	61.4	53.4	80.6	53.3	45.0
	RAG-2048	66.8	50.3	40.1	48.6	28.3	21.7	67.4	39.1	28.9	82.8	57.7	48.1	74.5	50.9	41.3
	RAG-4096	27.4	22.3	19.1	28.8	17.9	13.9	31.7	20.1	12.8	35.9	25.8	22.0	32.9	23.5	19.2
	Zep	60.2	23.9	20.0	43.8	24.2	19.3	53.7	30.5	20.4	66.9	45.5	40.0	61.6	36.9	30.9
	Mem0	56.9	39.2	33.2	47.9	23.7	17.7	68.2	40.1	30.3	71.4	48.6	42.0	66.3	43.5	36.5
	A-Mem	66.7	40.3	33.7	37.5	13.4	12.7	55.7	30.4	20.0	64.0	45.0	39.8	61.4	39.4	33.2
	Nemori	77.6	57.7	50.2	51.0	25.8	19.3	75.1	41.7	31.9	84.9	58.8	51.5	79.4	53.4	45.6
	Mem++	81.4	41.7	33.5	53.0	29.0	22.1	74.8	41.9	32.0	87.0	62.1	54.8	81.5	52.1	44.2
	Mem ^g ++	81.1	41.8	33.6	50.5	27.9	22.8	71.8	41.2	32.0	86.4	62.0	55.0	80.4	51.9	44.3
gpt-4o-mini	Full Context	56.2	44.1	36.1	48.6	24.5	17.2	66.8	35.4	26.1	83.0	53.1	44.7	72.3	46.2	37.8
	RAG-2048	62.6	47.5	38.2	45.2	23.4	18.1	63.0	35.7	24.6	78.5	53.6	43.0	70.3	47.2	37.1
	RAG-4096	22.8	18.4	15.2	35.5	17.6	15.6	31.1	18.6	11.7	33.0	23.9	19.2	30.7	21.4	16.8
	Zep	58.9	44.8	38.1	39.6	22.9	15.7	50.5	27.5	19.3	63.2	39.7	33.7	58.5	37.5	30.9
	Mem0	50.4	44.4	37.6	40.6	27.1	19.4	60.3	34.3	25.2	68.1	44.4	37.7	61.3	41.5	34.2
	A-Mem	54.2	38.1	33.8	22.9	9.0	8.6	43.6	24.0	18.8	58.2	35.6	29.2	52.5	32.4	27.0
	Nemori	71.0	56.7	46.6	44.8	20.8	15.1	65.3	36.5	25.6	82.1	54.4	43.2	74.4	49.5	38.5
	Mem++	76.9	56.9	46.5	47.3	24.2	17.8	68.5	39.0	27.5	84.0	59.7	47.9	77.4	53.1	42.0
	Mem ^g ++	75.6	55.3	44.5	46.2	24.6	18.8	65.8	37.2	25.3	84.1	58.9	47.3	76.7	52.1	41.0

Table 3: Performance on LongMemEvals (Wu et al., 2024) by question type. LLM-judge accuracy is reported. Bold indicates the best performance. Underline indicates the second best performance.

Question Type		Full Context	Zep	Nemori	Mem++	Mem ^g ++
gpt-4o-mini	Single-session preference	6.7	20.0	<u>46.7</u>	<u>46.7</u>	50.0
	Single-session assistant	89.3	80.4	83.9	<u>94.6</u>	96.4
	Temporal reasoning	42.1	62.4	<u>61.7</u>	<u>56.7</u>	56.7
	Multi-session	38.3	57.9	51.1	63.4	<u>62.0</u>
	Knowledge update	78.2	83.3	61.5	<u>84.3</u>	85.2
	Single-session user	78.6	<u>92.9</u>	88.6	98.4	98.4
	Average	55.0	68.0	64.2	<u>72.2</u>	72.4
gpt-4.1-mini	Single-session preference	16.7	22.5	86.7	47.8	<u>53.3</u>
	Single-session assistant	98.2	83.1	92.9	<u>94.6</u>	<u>94.6</u>
	Temporal reasoning	60.2	64.5	72.2	<u>68.5</u>	<u>69.0</u>
	Multi-session	51.1	57.6	55.6	56.1	<u>56.8</u>
	Knowledge update	76.9	<u>83.1</u>	79.5	89.9	89.9
	Single-session user	85.7	<u>96.3</u>	90.0	100.0	100.0
	Average	65.6	69.4	74.6	<u>74.7</u>	75.3

Zep to keeping every original document in the store with its date and author. In contrast, Mem0 overwrites a fact on conflict, which likely explains its Supersession scores of 40.8 and 32.3.

Competitive Performance on Conversational Memory. Although Mem++ targets organizational records, it also achieves the best average LLM-judge score on LoCoMo with both answerers. With gpt-4.1-mini, it reaches 81.5, surpassing Nemori by 2.1 points and Full Context by 0.9 points. With gpt-4o-mini, it reaches 77.4, exceeding Nemori by 3.0 points and Full Context by 5.1 points. The largest LLM-judge gain over Nemori is on Temporal Reasoning, where Mem++ scores 81.4 and 76.9, which is 3.8 and 5.9 points higher. We attribute this to the recency slots, which reserve part of the context for the latest-dated matching documents.

Performance on Long Contexts. LongMemEvals conversations exceed 100K tokens, an order of magnitude more than LoCoMo (Nan et al., 2025). On this benchmark, Mem^g++ achieves the best average with both answerers. It reaches 72.4 with gpt-4o-mini, 4.4 points above Zep, and 75.3 with gpt-4.1-mini, 0.7 points above Nemori. Mem++ ranks second with 72.2 and 74.7, and both variants exceed Full Context by at least 9.1 points. Moreover, the two variants hold the top two positions on Knowledge Update and Single-session User with both answerers.

Where Mem++ Trails. Mem++ does not lead every category. On OrgMemBench, RAG exceeds Mem++ on Contradiction by 28.2 and 27.4 points and on Bi-temporal with gpt-4.1-mini by 16.7 points. In addition, every method stays below 23 on Justification Chain. On LongMemEvals,

Table 4: Ablation with claude-sonnet-4-6 as the answerer. Parentheses give the change from Mem++.

Variant	OrgMemBench	LoCoMo	LongMemEvals
Mem++	57.0	85.7	87.9
w/o vector leg	21.3 (−35.7)	76.7 (−9.1)	11.9 (−76.0)
w/o lexical leg	57.2 (+0.2)	85.2 (−0.5)	88.7 (+0.7)
w/o tag leg	55.0 (−2.0)	—	—
w/ consolidation	55.6 (−1.3)	85.4 (−0.3)	86.9 (−1.1)
w/ fact index	55.2 (−1.7)	83.1 (−2.6)	—
Mem ^g ++	55.3 (−1.7)	84.8 (−0.9)	86.0 (−2.0)

Mem++ trails on Temporal Reasoning with both answerers, and Nemori leads Single-session Preference with gpt-4.1-mini (86.7 vs. 47.8). We attribute the preference gap to Nemori’s distillation, which can infer preferences that are rarely seen. On LoCoMo, Mem++ also trails Nemori on Temporal F1 with gpt-4.1-mini (41.7 vs. 57.7). However, the two methods are level with gpt-4o-mini (56.9 vs. 56.7), suggesting that the gap stems from answers phrasing rather than retrieval.

Limited Gain from the Graph. The graph in Mem^g++ changes the overall score by at most 1.1 points in Tables 1–3. Since the graph adds two LLM calls per document at write and one per question at read, we consider Mem++ the better trade-off between accuracy and cost.

4.3 ABLATION STUDY

Table 4 reports the ablation with claude-sonnet-4-6 as the answerer with the same gpt-4o-mini judge.

Retrieval Legs. Removing the vector leg lowers the score by 35.7 and 76.0 points on OrgMemBench and LongMemEvals but only 9.1 on LoCoMo, where the lexical leg matches any query term rather than all of them (Table 6). Removing the lexical leg changes the score by at most 0.7 points. The tag leg indexes only author labels and leaves the retrieved context byte-identical on the 73 OrgMemBench questions we compared, so its 2.0-point drop is run-to-run noise.

Write-Time Processing. Mem++ performs as well without consolidation or the fact index as with them, which shows that its accuracy does not depend on LLM calls at write time. Indexing each original document whole also scores 2.6 points higher on LoCoMo than indexing extracted facts, even though the answerer reads the same original documents in both variants. These results support the non-destructive design of Mem++, which keeps each document intact and leaves the choice between versions to read time.

Entity Graph. Mem^g++ lowers the score by 0.9 and 2.0 points on LoCoMo and LongMemEvals, while it led Mem++ on LongMemEvals by at most 0.6 points with the other answerers. Its gain is therefore not consistent across answerers. On OrgMemBench, the answerer reads only the first 40,000 characters and never sees the graph triples, so this row does not measure the graph.

5 CONCLUSION

We have argued for a non-destructive view of organizational memory, where each document is kept whole rather than distilled at write time. Mem++ follows this view by keeping every version in one store and deferring the choice between versions to read time. Experiments show that this design outperforms write-time extraction on OrgMemBench, achieves the best average LLM-judge score on LoCoMo and remains competitive on LongMemEvals. Ablations further show that semantic retrieval over original documents drives these results, whereas consolidation, fact extraction and an entity graph add cost without a consistent gain. These findings suggest that keeping the original documents and choosing among them at read time is a strong baseline for organizational memory. We acknowledge that few public benchmarks target organizational memory, so our organizational evaluation rests on the 73 questions that OrgMemBench asks about one synthetic organization. In future work, we will collect more diverse organizational records and test Mem++ on them to improve the system in this setting.

AI USE STATEMENT

In this work, the concept and architecture of Mem++ are the authors' own work and design. We used generative AI tools for partial help with generating the avatars using GPT-2, which we used later in our manually designed flowcharts and figures. We also utilized agentic AI tools to help cut the paper text to meet the page requirements. However, AI tools assisted the authors with setting the experiments and running scripts, three authors have been reviewing and monitoring the whole proposed memory system with result verifications over a period of roughly 4 months. We emphasize that the authors designed Mem++ themselves, based on their review of prior work and the limitations they identified in existing systems. Some parts of the text, such as the ablation study, were drafted by an LLM and then reviewed by the authors. The cited works are valid systems that we compare with in the first place and well known. We have reviewed all AI-assisted work and take full responsibility for the content of this paper.

REPRODUCIBILITY STATEMENT

We will release the full system, including code and experiments will be open sourced for the community upon publication.

REFERENCES

- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 758–759, 2009.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitanaky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, et al. Lightmem: Lightweight and efficient memory-augmented generation. In *International Conference on Learning Representations*, volume 2026, pp. 98706–98729, 2026.
- Jack Gardner. Orgmembench: A benchmark for long-horizon organizational memory in ai agents. <https://github.com/JackCGardner/OrgMemBench>, 2026. Preprint.
- Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems*, 37:59532–59569, 2024.
- Zhengjun Huang, Zhoujin Tian, Qintian Guo, Fangyuan Zhang, Yingli Zhou, Di Jiang, Zeyang Xie, and Xiaofang Zhou. Licomemory: Lightweight and cognitive agentic memory for efficient long-term reasoning. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 36842–36858, 2026.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp. 6769–6781, 2020.
- Sangyeop Kim, Yohan Lee, Sanghwa Kim, Hyunjong Kim, and Sungzoon Cho. Pre-storage reasoning for episodic memory: Shifting inference burden to memory for personalized dialogue. *arXiv preprint arXiv:2509.10852*, 2025.
- Lukas Kirchdorfer, Adrian Rebmam, Christian Warmuth, Timotheus Kampik, Theiss Heilker, and Gregor Berg. Organizational memory for agentic business process execution. *arXiv preprint arXiv:2607.03228*, 2026.

- LangChain AI. LangMem: Long-term memory SDK for LLM agents. <https://langchain-ai.github.io/langmem/>, 2025. Accessed: 2025-11-20.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.
- Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*, 2025.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870, 2024.
- Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. Nemori: Self-organizing agent memory inspired by cognitive science. *arXiv e-prints*, pp. arXiv–2508, 2025.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- James P Walsh and Gerardo Rivera Ungson. Organizational memory. *Academy of management review*, 16(1):57–91, 1991.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604, 2026.
- Guancheng Zeng, Xueyi Chen, Jiawang Hu, Shaohua Qi, Yaxuan Mao, Zhantao Wang, Yifan Nie, Shuang Li, Qiuyang Feng, Pengxu Qiu, et al. Routine: A structural planning framework for llm agent system in enterprise. *arXiv preprint arXiv:2507.14447*, 2025.
- Guibin Zhang, Muxin Fu, Kun Wang, Frank Wan, Miao Yu, and Shuicheng Yan. G-memory: Tracing hierarchical memory for multi-agent systems. *Advances in Neural Information Processing Systems*, 38:12988–13018, 2026.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 19724–19731, 2024.

Mem++ Appendix

A METHOD DETAILS

Algorithm 1 summarizes the write and read paths of Mem++, and Algorithm 2 summarizes the optional consolidation operator. Here f_{LLM} is a language model, f_{ans} is the answering model and $\mathcal{K}$ is the entity graph of Mem^{g++}. Each $\mathcal{P}$ is a fixed prompt, $\parallel$ denotes concatenation and τ is the similarity threshold of consolidation.

Algorithm 1 Mem++ Write and Read Paths

Require: Document stream $\mathcal{D}$, question q , optional as-of date θ

Ensure: Store $\mathcal{R}$, evidence E , answer $\hat{y}$

```

% — Write path (§3.2) —
1: for each document  $(x, a, t) \in \mathcal{D}$  do
2:   if  $x$  is empty or exactly repeats an active row in the author scope of  $a$  then
3:     continue ▷ no row is added
4:   end if
5:    $\mathbf{e} \leftarrow h(x)$  ▷ no generative call
6:    $r \leftarrow \langle x, a, t, \mathbf{e}, \sigma \rangle$  with  $\sigma$  set to active and current
7:    $\mathcal{R} \leftarrow \mathcal{R} \cup \{r\}$ 
8:   Index  $x$  for lexical search,  $a$  as a tag and  $\mathbf{e}$  for vector search
9:   if running Memg++ then
10:     $\mathcal{K} \leftarrow \mathcal{K} \cup f_{\text{LLM}}(\mathcal{P}_{\text{triple}} \parallel x)$  ▷ two LLM calls per document or chunk
11:   end if
12: end for
13: if consolidation is enabled then
14:    $\mathcal{C}(\mathcal{R}, \tau)$  ▷ Algorithm 2, off in headline runs
15: end if
% — Read path (§3.3) —
16:  $\mathcal{S}_\theta \leftarrow \{r \in \mathcal{R} \mid \text{active}(\sigma_r), t_r \leq \theta\}$  ▷ time condition only if  $\theta$  is given
17: for  $\ell \in \{\text{lex}, \text{tag}, \text{vec}\}$  do
18:    $L_\ell^\theta \leftarrow$  at most  $C$  rows of  $\mathcal{S}_\theta$  ranked by index  $\ell$  for  $q$ 
19: end for
20: Score each  $r \in \bigcup_\ell L_\ell^\theta$  by  $\text{RRF}(r) = \sum_{\ell: r \in L_\ell^\theta} w_\ell / (\gamma + \text{rank}_\ell(r))$ 
21:  $E \leftarrow$  the first  $k - 3$  rows by score, with ties broken by ingest recency
22:  $E \leftarrow E \parallel$  the three latest-dated remaining matched rows ▷ undated rows use ingest time
23: Fill any missing reserve slot with the most recently ingested unmatched rows
24: if running Memg++ then
25:    $V_q \leftarrow f_{\text{LLM}}(\mathcal{P}_{\text{entity}} \parallel q)$  ▷ entities named in  $q$ 
26:    $E \leftarrow E \parallel$  at most five triples of  $\mathcal{K}$  reached from  $V_q$  ▷ no as-of filter
27: end if
28:  $\hat{y} \leftarrow f_{\text{ans}}(q \parallel E)$  with rows in rank order, each preceded by its date and author where recorded

```

Consolidation Operator ($\mathcal{C}$). Unlike memories that run a language model on every incoming document (Chhikara et al., 2025; Rasmussen et al., 2025; Xu et al., 2026), $\mathcal{C}$ runs between writes and queries over documents already stored. It spends a model call only where two documents likely describe the same fact. The operator first greedily groups current rows, where each group takes the rows whose embeddings have a cosine similarity of at least τ to its first row. It then asks a language model what changed within each group B and assigns the group a label

$$y(B) \in \{\text{CONFLICT}, \text{RESTATEMENT}, \text{DISTINCT}\}, \quad (6)$$

where a conflict means one fact took a new value over time, a restatement means the rows say the same thing in different words, and distinct means the rows only look alike. In a conflict, the row with the latest event date stays current. Each older row keeps its text and date and is marked in σ as superseded by that row, and it therefore remains retrievable with its own date. In a restatement, a second model call writes one merged row of at most 220 tokens. This row takes the earliest date in

Algorithm 2 Consolidation Operator $\mathcal{C}$ **Require:** Store $\mathcal{R}$, similarity threshold τ **Ensure:** Updated states σ and merged rows

```

1: Group current rows greedily by cosine  $\geq \tau$  to the first row of each group
2: for each group  $B$  with more than one row do
3:    $y(B) \leftarrow f_{\text{LLM}}(\mathcal{P}_{\text{label}} \parallel B)$   $\triangleright$  skip  $B$  if the call fails
4:   if  $y(B) = \text{CONFLICT}$  then
5:      $r^* \leftarrow$  the row of  $B$  with the latest event date
6:     Mark every other row of  $B$  as superseded by  $r^*$  in  $\sigma$   $\triangleright$  row stays active
7:   else if  $y(B) = \text{RESTATEMENT}$  then
8:      $x' \leftarrow f_{\text{LLM}}(\mathcal{P}_{\text{merge}} \parallel B)$   $\triangleright$  at most 220 tokens
9:     Add and index the active row  $\langle x', A_B, \min_{r \in B} t_r, h(x'), \sigma \rangle$   $\triangleright A_B$  is every author of  $B$ 
10:    Mark the rows of  $B$  as archived in  $\sigma$   $\triangleright$  removed from retrieval
11:   else
12:     Leave  $B$  untouched  $\triangleright$  DISTINCT
13:   end if
14: end for

```

B and lists every original author. The originals are marked as archived in σ , which removes them from retrieval. These merged rows are the only rows in the store whose text a model has written. Because of this earliest date, a merged row can carry text from later rows past the as-of date θ . This affects only the consolidation ablation, since consolidation is off by default. A distinct group is left untouched, and so is any group whose labeling call fails. The cost is therefore at most two model calls per group rather than one per document.

B ADDITIONAL EXPERIMENTAL DETAILS

B.1 BASELINES

Full Context places the whole history in the prompt of the answering model and uses no external memory.

RAG (Lewis et al., 2020; Karpukhin et al., 2020) splits the history into fixed-size chunks and returns the chunks whose embeddings are closest to the question. On LoCoMo, the suffix gives the chunk size in tokens. Unlike Mem++, it can split one document across several chunks.

Mem0 (Chhikara et al., 2025) uses an LLM to extract salient facts from each new message and compares each fact with the most similar stored memories. An LLM then adds, updates or deletes a memory or leaves the store unchanged, so a newer fact can overwrite an older one.

Zep (Rasmussen et al., 2025) builds a temporal knowledge graph with its Graphiti engine. It extracts entities and facts from each episode and stores each fact as an edge with the period over which it held. When a later fact contradicts an edge, Zep invalidates that edge. At query time it combines semantic, keyword and graph search over facts, entities and communities.

A-Mem (Xu et al., 2026) follows the Zettelkasten method and stores each interaction as a note with keywords, tags and a context description written by an LLM. It links each new note to related notes and can update the attributes of older notes as new ones arrive.

Nemori (Nan et al., 2025) segments a conversation into coherent episodes and rewrites each episode as a narrative. It then distills semantic knowledge by predicting a new episode from existing memory and storing what the prediction missed.

gbrain (Gardner, 2026) is an open-source memory system that combines keyword, vector and graph search over a store extracted from its documents. We report it only on OrgMemBench, whose paper also evaluates it.

B.2 EVALUATION METRICS

LLM-Judge Score. Our primary metric asks gpt-4o-mini to judge each generated answer against the gold answer. On LoCoMo and LongMemEvals the judge labels each answer as correct or wrong. The score is the share of correct answers,

$$\text{LLM} = \frac{100}{N} \sum_{i=1}^N \mathbf{1}[J(q_i, a_i, \hat{a}_i) = \text{CORRECT}], \quad (7)$$

where J is the judge and N is the number of questions q_i with gold answers a_i and generated answers $\hat{a}_i$. LoCoMo uses the Mem0 judge prompt (Chhikara et al., 2025) over 1,520 questions. On LongMemEvals we use one prompt per question type following Zep (Rasmussen et al., 2025) and score our runs on its 470 non-abstention questions. Published baseline scores on LongMemEvals cover all 500 questions. On OrgMemBench the 73 questions carry rubrics with 247 weighted facets in total. In place of the default judge of the benchmark, gpt-4o-mini grades each answer against every facet of its rubric at temperature zero. This gives a question score $s_i \in [0, 1]$ that rewards partial coverage, and the overall score is $\frac{100}{N} \sum_i s_i$. We judge each of our runs three times and report the mean.

F1 Score. F1 is the harmonic mean of token-level precision P and recall R between the generated and gold answers after standard answer normalization,

$$\text{F1} = \frac{2PR}{P+R}, \quad P = \frac{|\hat{T} \cap T|}{|\hat{T}|}, \quad R = \frac{|\hat{T} \cap T|}{|T|}, \quad (8)$$

where $\hat{T}$ and T are the token multisets of the generated and gold answers.

BLEU-1. BLEU-1 is the unigram precision of the generated answer against the gold answer, scaled by a brevity penalty,

$$\text{BLEU-1} = \text{BP} \cdot \frac{\sum_w \min(c_{\hat{a}}(w), c_a(w))}{\sum_w c_{\hat{a}}(w)}, \quad \text{BP} = \begin{cases} 1 & \text{if } |\hat{a}| > |a|, \\ e^{1-|a|/|\hat{a}|} & \text{otherwise,} \end{cases} \quad (9)$$

where $c_x(w)$ counts word w in answer x and $|x|$ is the length of x in words. We report F1 and BLEU-1 on LoCoMo only. Both reward exact wording and can penalize a correct answer phrased differently, which is why the LLM-judge score is our primary metric.

B.3 IMPLEMENTATION DETAILS

Table 6 lists the retrieval settings that differ across benchmarks. All runs share the fusion constant $\gamma = 60$, $k = 50$ retrieved rows and three recency slots. A disjunctive lexical index returns rows that match any query term, whereas a conjunctive one returns only rows that contain every term. On OrgMemBench the encoder runs locally and embeds at most 256 word pieces of each document, while the stored text and the lexical index keep the full document. Only the 15 Audit Replay questions of OrgMemBench carry an as-of date, so the date bound applies only to them. On OrgMemBench the benchmark passes only the first 40,000 characters of the retrieved context to the answering model, and these hold about 19 of the 50 rows. The recency slots and the Mem^{g++} triples come after the fused rows, so they never reach the answering model on this benchmark. Mem^{g++} calls gpt-4o-mini twice per document at write and once per question at read. On the conversational benchmarks, its write calls run per ten-turn chunk instead of per document.

B.4 SENSITIVITY TO THE NUMBER OF RETRIEVED ROWS

Figure 4 and Table 7 vary the number of retrieved rows k on OrgMemBench with both answering models, while all other settings follow Section B.3. With gpt-4.1-mini, the score rises from 40.5 at $k = 5$ to 56.5 at $k = 30$ and then stays between 56.5 and 57.0 up to $k = 100$. With gpt-4o-mini, it rises from 36.6 at $k = 5$ to between 44.2 and 48.7 for every $k \geq 10$, with no clear trend beyond $k = 15$. The plateau is partly mechanical, because the benchmark passes only the first 40,000 characters of the context to the answering model, which hold about 19 rows and at most 25 (Section B.3). For $k \geq 30$ the fused rows alone exceed this budget, and their order does not depend

Table 5: Category distributions of the OrgMemBench (Gardner, 2026), LoCoMo (Maharana et al., 2024) and LongMemEvals (Wu et al., 2024) datasets.

OrgMemBench (medium)		LoCoMo		LongMemEvals	
Category	Count	Category	Count	Category	Count
C1 Supersession	15	Single-hop	841	Knowledge update	72
C2 Provenance	15	Multi-hop	282	Multi-session	121
C3 Bi-temporal	7	Temporal	321	Temporal reasoning	127
C4 Audit replay	15	Open domain	96	Single-session user	64
C5 Justification	15	–	–	Single-session assistant	56
C6 Contradiction	6	–	–	Single-session preference	30
Total	73	Total	1,540	Total	470

Table 6: Retrieval settings that differ across benchmarks.

Setting	OrgMemBench	LoCoMo	LongMemEvals
Embedding model	all-MiniLM-L6-v2	text-embedding-3-small	text-embedding-3-small
Embedding size	384	1536	1536
Weights (w_{lex} , w_{tag} , w_{vec})	(1, 1, 4)	(1, 1, 2)	(1, 1, 4)
Candidates per index C	50	200	50
Lexical matching	conjunctive	disjunctive	conjunctive

on k since each index returns at most C candidates. The answering model therefore reads the same rows for every $k \geq 30$, and the changes across these settings are run-to-run variation, which reaches 4.4 points with gpt-4o-mini. Retrieved tokens, counted before the cut, grow almost linearly with k and stay far below the 18.7M tokens of Full Context.

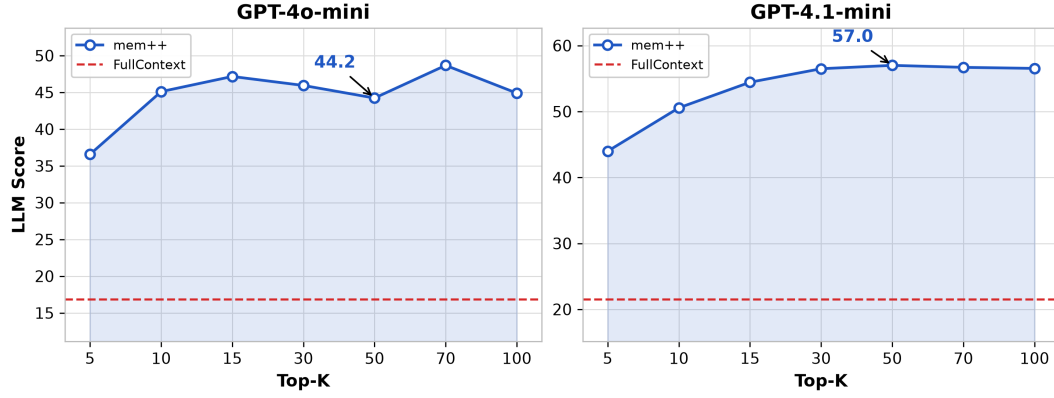


Figure 4: OrgMemBench score of Mem++ as the number of retrieved rows k varies. The dashed line is Full Context, and the dotted line marks $k = 50$ used in the main experiments.

Table 7: OrgMemBench score of Mem++ for each k . Retrieved tokens are summed over all questions and counted before the 40,000-character cut.

k	gpt-4o-mini	gpt-4.1-mini	Retrieved tokens
5	36.57	43.99	0.20M
10	45.10	50.56	0.41M
15	47.16	54.44	0.61M
30	45.94	56.49	1.22M
50	44.23	57.00	2.01M
70	48.67	56.70	2.77M
100	44.89	56.54	3.63M
Full Context	16.85	21.52	18.68M